Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO



2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



11

Relacionamento entre entidades e DTOs

Aula 41 a 44 - Desenvolvimento de Sistemas

Instrutor: Vitor Hugo Almeida de Sousa

Relacionamento entre entidades

Ligando um aluno a um livro

Crie Emprestimo.java no pacote model. Cada objeto representará um empréstimo, com seu próprio ID, a data da operação, um aluno e um livro.

@ManyToOne indica que vários empréstimos podem apontar para um mesmo aluno e que vários empréstimos podem apontar para um mesmo livro. Cada empréstimo, porém, referencia **um aluno e um livro**.

@JoinColumn define as colunas que guardarão essas referências na tabela emprestimo: aluno_id e livro_id. Essas colunas usam os IDs dos cadastros já existentes.

```
3 •   describe emprestimo;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
data_emprestimo	date	YES		NULL	
aluno_id	int	NO	MUL	NULL	
livro_id	int	NO	MUL	NULL	

Emprestimo.java

src > main > java > br > com > senai > teste > model > Emprestimo.java > Emprestimo

```java
package br.com.senai.teste.model;

import java.time.LocalDate;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "emprestimo")
public class Emprestimo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private LocalDate dataEmprestimo;

    @ManyToOne
    @JoinColumn(name = "aluno_id", nullable = false)
    private Aluno aluno;

    @ManyToOne
    @JoinColumn(name = "livro_id", nullable = false)
    private Livro livro;

    public Emprestimo() {
    }

    public Integer getId() {
        return id;
    }

    public LocalDate getDataEmprestimo() {
        return dataEmprestimo;
    }

    public void setDataEmprestimo(LocalDate dataEmprestimo) {
        this.dataEmprestimo = dataEmprestimo;
    }

    public Aluno getAluno() {
        return aluno;
    }

    public void setAluno(Aluno aluno) {
        this.aluno = aluno;
    }

    public Livro getLivro() {
        return livro;
    }

    public void setLivro(Livro livro) {
        this.livro = livro;
    }
}
```

Criando o EmprestimoRepository

Preparando a comunicação com a tabela emprestimo

A tabela emprestimo já foi criada a partir do Model. Agora, precisamos de um Repository para consultar e gravar seus registros, seguindo a mesma estrutura utilizada para alunos e livros.

No pacote repository:

- crie a interface EmprestimoRepository.java

Emprestimo informa qual entidade será manipulada, e Integer corresponde ao tipo do seu ID. A interface já disponibiliza operações como save(), findAll() e findById().

O Repository consegue gravar um empréstimo, mas ainda precisamos definir **como encontrar o aluno e o livro escolhidos pelo cliente** antes de chamar save().

Esse será o papel do Service no próximo passo.

EmprestimoRepository.java

src > main > java > br > com > senai > teste > repository > EmprestimoRepository.java

```java
package br.com.senai.teste.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import br.com.senai.teste.model.Emprestimo;

public interface EmprestimoRepository
        extends JpaRepository<Emprestimo, Integer> {

}

```

Criando o EmprestimoService

Localizando o aluno e o livro antes de gravar

Para criar um empréstimo, o cliente informará o ID de um aluno e o ID de um livro. O Service consultará os dois cadastros antes de associá-los a um novo Emprestimo.

No pacote service:

- crie EmprestimoService.java

Os métodos findById() procuram os registros já existentes. Se algum dos IDs não for encontrado, o Service devolve um Optional vazio e não grava o empréstimo.

Quando ambos existem, o Service cria um Emprestimo, associa os objetos encontrados, registra a data atual e usa save() para gravá-lo.

LocalDate.now() utiliza a data do computador em que a aplicação Spring Boot está executando. Neste primeiro exemplo, o cliente informa apenas os IDs; a aplicação preenche a data automaticamente.

EmprestimoService.java

src > main > java > br > com > senai > teste > service > EmprestimoService.java > ...

```java
package br.com.senai.teste.service;

import java.time.LocalDate;
import java.util.Optional;

import org.springframework.stereotype.Service;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.model.Emprestimo;
import br.com.senai.teste.model.Livro;
import br.com.senai.teste.repository.AlunoRepository;
import br.com.senai.teste.repository.EmprestimoRepository;
import br.com.senai.teste.repository.LivroRepository;

@Service
public class EmprestimoService {

    private final EmprestimoRepository emprestimoRepository;
    private final AlunoRepository alunoRepository;
    private final LivroRepository livroRepository;

    public EmprestimoService(
            EmprestimoRepository emprestimoRepository,
            AlunoRepository alunoRepository,
            LivroRepository livroRepository) {

        this.emprestimoRepository = emprestimoRepository;
        this.alunoRepository = alunoRepository;
        this.livroRepository = livroRepository;
    }

    public Optional<Emprestimo> cadastrar(
            Integer alunoId, Integer livroId) {

        Optional<Aluno> aluno = alunoRepository.findById(alunoId);
        Optional<Livro> livro = livroRepository.findById(livroId);

        if (aluno.isEmpty() || livro.isEmpty()) {
            return Optional.empty();
        }

        Emprestimo emprestimo = new Emprestimo();
        emprestimo.setAluno(aluno.get());
        emprestimo.setLivro(livro.get());
        emprestimo.setDataEmprestimo(LocalDate.now());

        return Optional.of(emprestimoRepository.save(emprestimo));
    }
}
```

O que é um DTO?

Saiba Mais!



DTO significa Data Transfer Object, ou objeto de transferência de dados.
Essa classe representa os dados que a API espera receber na requisição. Ela não cria uma tabela no banco, pois não possui a anotação @Entity.

Quando NÃO utilizar um DTO?

Cadastro de aluno sem DTO

No cadastro de aluno, o JSON possui os mesmos campos existentes no model:

```
{
    "nome": "Ana",
    "email": "ana@email.com"
}
```

Esses dados podem ser convertidos diretamente para um objeto Aluno:

```
@PostMapping
public ResponseEntity<Aluno> cadastrar(
        @RequestBody Aluno aluno) {
```

X

Quando utilizar um DTO?

Quando o aluno precisaria de um DTO?

Um DTO seria útil se os dados enviados fossem diferentes daqueles armazenados diretamente no model.

Imagine um cadastro que também recebesse a confirmação do e-mail:

```
{
    "nome": "Ana",
    "email": "ana@email.com",
    "confirmacaoEmail": "ana@email.com"
}
```

O campo confirmacaoEmail é necessário apenas para validar o cadastro. Ele não precisa ser armazenado na tabela de alunos.

Nesse caso, poderíamos criar:

```
public class AlunoRequest {

    private String nome;
    private String email;
    private String confirmacaoEmail;
}
```

Firjan SENAI

Criando o DTO

Dados para cadastrar o empréstimo

O cadastro de um empréstimo receberá o ID do aluno e o ID do livro. Embora esses valores sejam números, eles não pertencem diretamente ao Model Emprestimo, que possui objetos dos tipos Aluno e Livro.

No pacote principal do projeto, crie o pacote:

- dto

No pacote principal do projeto, crie o pacote:

- EmprestimoRequest.java

EmprestimoRequest.java

src > main > java > br > com > senai > teste > dto > EmprestimoR

```java
package br.com.senai.teste.dto;

public class EmprestimoRequest {

    private Integer alunoId;
    private Integer livroId;

    public EmprestimoRequest() {
    }

    public Integer getAlunoId() {
        return alunoId;
    }

    public void setAlunoId(Integer alunoId) {
        this.alunoId = alunoId;
    }

    public Integer getLivroId() {
        return livroId;
    }

    public void setLivroId(Integer livroId) {
        this.livroId = livroId;
    }
}
```

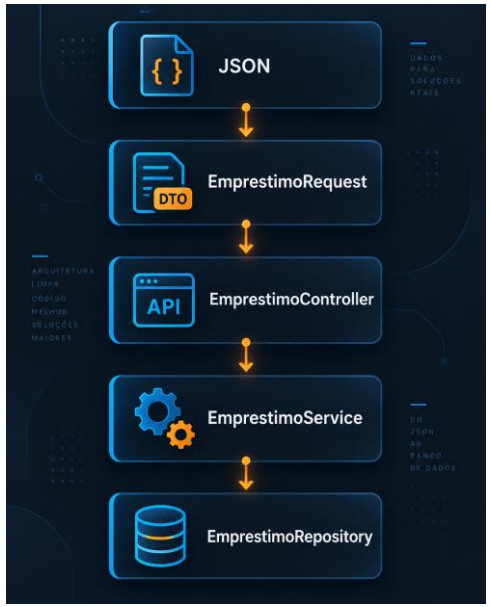
Controller de empréstimos

Recebendo os IDs por meio do DTO

A requisição de cadastro enviará o ID do aluno e o ID do livro. O Spring utilizará a classe EmprestimoRequest para transformar o JSON recebido em um objeto Java.

No pacote controller:

- crie EmprestimoController.java



EmprestimoController.java

```java
package br.com.senai.teste.controller;

import java.util.Optional;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.dto.EmprestimoRequest;
import br.com.senai.teste.model.Emprestimo;
import br.com.senai.teste.service.EmprestimoService;

@RestController
@RequestMapping("/emprestimos")
public class EmprestimoController {

    private final EmprestimoService emprestimoService;

    public EmprestimoController(EmprestimoService emprestimoService) {
        this.emprestimoService = emprestimoService;
    }

    @PostMapping
    public ResponseEntity<Emprestimo> cadastrar(
            @RequestBody EmprestimoRequest dados) {

        Integer alunoId = dados.getAlunoId();
        Integer livroId = dados.getLivroId();

        if (alunoId == null || livroId == null) {
            return ResponseEntity.badRequest().build();
        }

        Optional<Emprestimo> emprestimo = emprestimoService.cadastrar(
                alunoId,
                livroId);

        if (emprestimo.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(emprestimo.get());
    }

}
```

Cadastro de empréstimo

Do Postman às três tabelas do banco

Reinicie a aplicação para aplicar as alterações.

Antes de enviar a requisição, confirme que existe pelo menos um aluno e um livro cadastrados. Consulte suas rotas no Postman e anote os IDs que serão usados:

- GET http://localhost:8080/alunos
- GET http://localhost:8080/livros

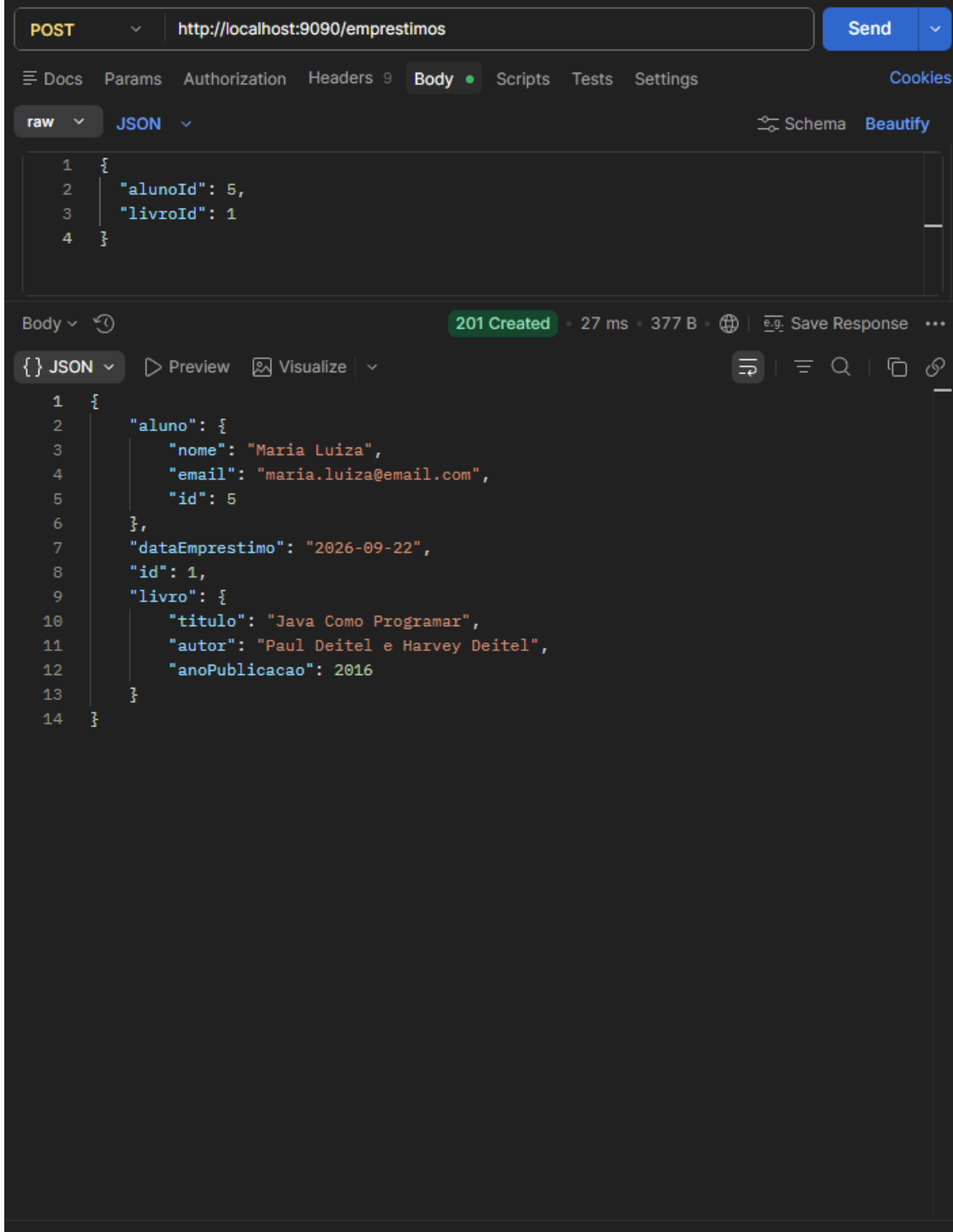
Depois, crie uma requisição POST para:

- http://localhost:8080/emprestimos

Em Body → raw → JSON, envie os IDs encontrados. Por exemplo:

```
{
  "alunoId": 1,
  "livroId": 2
}
```

Resultado esperado: *201 Created. A resposta apresentará o ID do empréstimo, a data preenchida pela aplicação e os dados do aluno e do livro associados.*



Validação

Teste dos outros resultados:

- Envie o JSON sem livroId: a resposta deverá ser 400 Bad Request.
- Envie um alunoId que não exista: a resposta deverá ser 404 Not Found, sem criar um empréstimo.

Atenção



Se o cadastro retornar 500, consulte primeiro o console do Spring Boot. Verifique especialmente se os IDs existem e se a tabela emprestimo foi criada com as colunas de relacionamento.

Consulta de empréstimos

Visualizando os registros e seus vínculos

O cadastro já relaciona um aluno e um livro. Agora, vamos criar uma rota para consultar todos os empréstimos e observar como essas informações aparecem na resposta JSON.

No EmprestimoService.java:

- adicione a importação
- Depois, acrescente o método

No EmprestimoController.java:

- adicione as importações
- acrescente o método

findAll() consulta os registros da tabela emprestimo. Como cada registro contém referências a um Aluno e a um Livro, a resposta JSON apresentará o empréstimo com os dados das entidades associadas.

EmprestimoService.java

```java
import java.util.List;
```

```java
public List<Emprestimo> listar() {
    return emprestimoRepository.findAll();
}
```

EmprestimoController.java

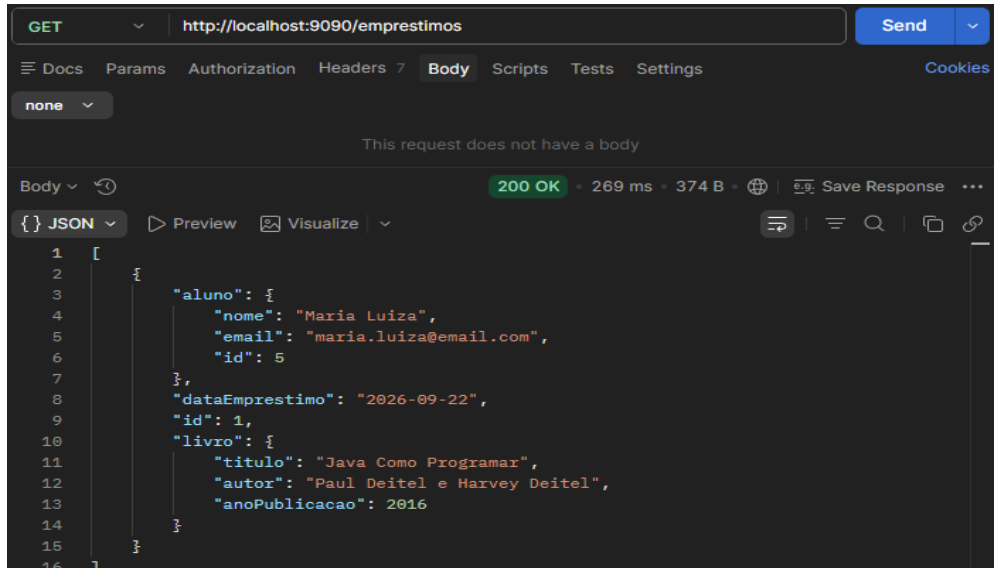
```java
import java.util.List;
```

```java
import org.springframework.web.bind.annotation.GetMapping;
```

```java
@GetMapping
public ResponseEntity<List<Emprestimo>> listar() {
    List<Emprestimo> emprestimos = emprestimoService.listar();

    return ResponseEntity.ok(emprestimos);
}
```

Teste no Postman: GET http://localhost:8080/emprestimos



Consulta por ID

Localizando um empréstimo específico

No EmprestimoService.java:

- acrescente o método

No EmprestimoController.java:

- adicione as importações
- acrescente o método

@PathVariable recebe o número informado após /emprestimos/. O Service consulta esse ID no Repository. Se o registro existir, a API devolve o empréstimo com 200 OK; caso contrário, devolve 404 Not Found.

EmprestimoService.java

```java
public Optional<Emprestimo> buscarPorId(Integer id) {
    return emprestimoRepository.findById(id);
}
```

EmprestimoController.java

```java
import org.springframework.web.bind.annotation.PathVariable;
```

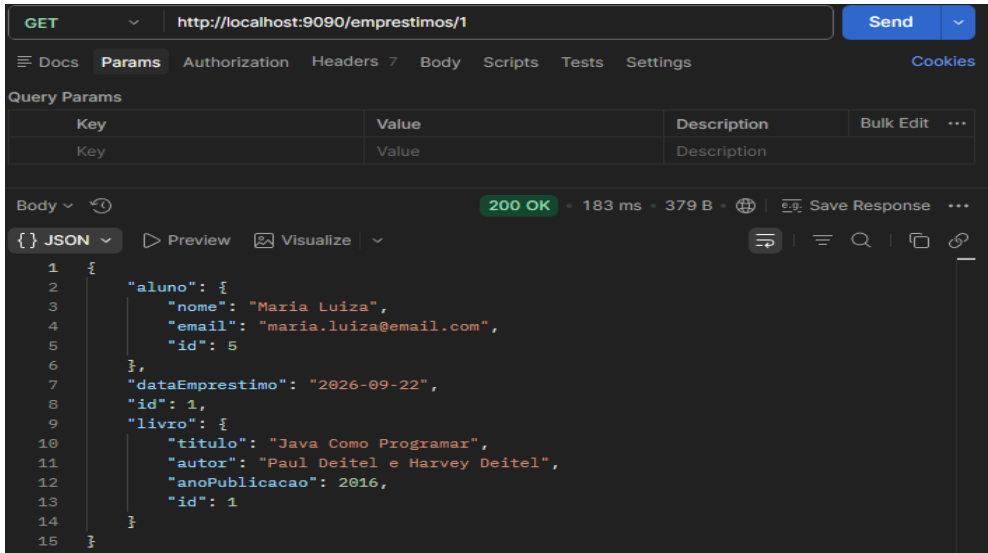
```java
@GetMapping("/{id}")
public ResponseEntity<Emprestimo> buscarPorId(
        @PathVariable Integer id) {

    Optional<Emprestimo> emprestimo = emprestimoService.buscarPorId(id);

    if (emprestimo.isPresent()) {
        return ResponseEntity.ok(emprestimo.get());
    }

    return ResponseEntity.notFound().build();
}
```

Teste no Postman: GET http://localhost:8080/emprestimos/1



Firjan SENAI